

Como fazer um jogo igual ao Termo com HTML, CSS e JavaScript

Guia completo — do zero ao código funcionando. Baseado na estrutura passo a passo do exemplo enviado, mas adaptado para um projeto em HTML, CSS e JavaScript puro.[file:1]
[code_file:2]

1. configuração

Criar os arquivos do projeto

Comece criando uma pasta para o projeto e, dentro dela, três arquivos: `index.html`, `style.css` e `script.js`. [code_file:2]

Essa separação ajuda a organizar melhor o projeto: o HTML cuida da estrutura, o CSS do visual e o JavaScript da lógica do jogo. [code_file:2]

```
termo/  
├─ index.html  
├─ style.css  
└─ script.js
```

2. configuração

Montar o HTML base

No arquivo `index.html`, crie a estrutura principal da página com um título, uma área para o tabuleiro, uma área para o teclado virtual e uma área para mensagens ou modal final. [code_file:2]

O essencial é deixar elementos com `id` para o JavaScript conseguir manipular depois. [code_file:2]

```
<body>  
  <header>  
    <h1>Termo</h1>  
  </header>  
  
  <div id="toast-container"></div>  
  <div id="board"></div>  
  <div id="keyboard"></div>  
  
  <div id="modal-overlay">  
    <div id="modal">  
      <h2 id="modal-title"></h2>
```

```
<div id="modal-word"></div>
<p id="modal-msg"></p>
<button id="btn-restart">Jogar Novamente</button>
</div>
</div>

<script src="script.js"></script>
</body>
```

3. configuração

Definir o visual geral no CSS

No arquivo `style.css`, defina as cores principais do jogo. O Termo normalmente usa fundo escuro, letras brancas, cinza para letras ausentes, amarelo para letras presentes e verde para letras corretas.[code_file:2]

Também vale criar variáveis CSS para facilitar futuras mudanças no estilo.[code_file:2]

```
:root {
  --color-bg: #121213;
  --color-text: #ffffff;
  --color-border: #3a3a3c;
  --color-correct: #538d4e;
  --color-present: #b59f3b;
  --color-absent: #3a3a3c;
}
```

4. código

Criar o tabuleiro do jogo

O tabuleiro do Termo tem 6 linhas e 5 colunas. Em vez de escrever tudo manualmente no HTML, o melhor é gerar essa grade com JavaScript.[code_file:2]

Cada célula pode ser uma `div` com a classe `tile`, e cada linha uma `div` com a classe `row`. [code_file:2]

```
function buildBoard() {
  const board = document.getElementById("board");
  board.innerHTML = "";

  for (let r = 0; r < 6; r++) {
    const row = document.createElement("div");
    row.className = "row";

    for (let c = 0; c < 5; c++) {
      const tile = document.createElement("div");
      tile.className = "tile";
      row.appendChild(tile);
    }
  }
}
```

```
    }

    board.appendChild(row);
  }
}
```

5. código

Estilizar as células do tabuleiro

Agora estilize cada quadradinho para ele ficar parecido com o jogo original. Use largura, altura, borda, alinhamento central e texto em maiúsculo.[code_file:2]

```
.row {
  display: flex;
  gap: 5px;
}

.tile {
  width: 62px;
  height: 62px;
  border: 2px solid #3a3a3c;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 32px;
  font-weight: bold;
  text-transform: uppercase;
  color: white;
}
```

6. código

Criar a lista de palavras e a palavra secreta

O jogo precisa de uma lista de palavras válidas com 5 letras. Depois disso, escolha uma palavra aleatória para ser a resposta da rodada.[code_file:2]

```
const WORD_LIST = ["CARRO", "PEDRA", "LIVRO", "PORTA", "FONTE"];
const secretWord = WORD_LIST[Math.floor(Math.random() * WORD_LIST.length)];
```

Se quiser deixar o jogo mais completo, aumente essa lista com várias palavras em português de 5 letras.[code_file:2]

7. código

Criar as variáveis de estado do jogo

O JavaScript precisa guardar algumas informações durante a partida: palavra atual digitada, tentativas já feitas, avaliações e se o jogo terminou.[code_file:2]

```
let guesses = [];  
let evaluations = [];  
let currentInput = "";  
let gameOver = false;  
const MAX_ATTEMPTS = 6;  
const WORD_LENGTH = 5;
```

Essas variáveis são o “estado” do jogo fora do React, controladas manualmente no JavaScript.[file:1][code_file:2]

8. código

Capturar teclas do teclado físico

Para permitir que o jogador digite normalmente, adicione um evento de teclado no documento e trate letras, Enter e Backspace.[code_file:2]

```
document.addEventListener("keydown", e => {  
  const key = e.key.toUpperCase();  
  
  if (key === "ENTER") {  
    submitGuess();  
  } else if (key === "BACKSPACE") {  
    currentInput = currentInput.slice(0, -1);  
    updateCurrentRow();  
  } else if (/^[A-Z]$/.test(key) && currentInput.length < WORD_LENGTH) {  
    currentInput += key;  
    updateCurrentRow();  
  }  
});
```

9. código

Mostrar a palavra atual na linha ativa

Sempre que o jogador digitar ou apagar uma letra, atualize visualmente apenas a linha da tentativa atual.[code_file:2]

```
function updateCurrentRow() {  
  const row = guesses.length;  
  
  for (let c = 0; c < WORD_LENGTH; c++) {
```

```

    const tile = document.getElementById(`tile-${row}-${c}`);
    tile.textContent = currentInput[c] || "";
  }
}

```

Para isso funcionar perfeitamente, é melhor criar IDs únicos para cada tile quando o tabuleiro for montado.[code_file:2]

10. código

Criar a lógica de verificação da tentativa

Essa é a parte principal do jogo. A função compara a palavra digitada com a palavra secreta e decide se cada letra está correta, presente ou ausente.[file:1][code_file:2]

A melhor forma é fazer em duas etapas: primeiro marcar as letras na posição certa; depois verificar letras existentes em posição errada.[file:1][code_file:2]

```

function evaluateGuess(guess, secret) {
  const result = Array(secret.length).fill("absent");
  const secretArr = secret.split("");
  const guessArr = guess.split("");

  guessArr.forEach((letter, i) => {
    if (letter === secretArr[i]) {
      result[i] = "correct";
      secretArr[i] = null;
      guessArr[i] = null;
    }
  });

  guessArr.forEach((letter, i) => {
    if (!letter) return;
    const idx = secretArr.indexOf(letter);
    if (idx !== -1) {
      result[i] = "present";
      secretArr[idx] = null;
    }
  });

  return result;
}

```

11. código

Enviar a tentativa ao apertar Enter

Quando o jogador apertar Enter, o jogo precisa verificar se a palavra tem 5 letras, avaliar o chute, salvar a tentativa e preparar a próxima linha.[code_file:2]

```
function submitGuess() {
  if (currentInput.length < WORD_LENGTH) {
    alert("Palavra muito curta!");
    return;
  }

  const evaluation = evaluateGuess(currentInput, secretWord);
  guesses.push(currentInput);
  evaluations.push(evaluation);
  currentInput = "";
}
```

Depois disso, você deve atualizar o tabuleiro e verificar vitória ou derrota.[code_file:2]

12. código

Pintar as letras com as cores corretas

Após avaliar a palavra, percorra os tiles da linha enviada e aplique classes ou atributos para mudar a cor de cada célula.[code_file:2]

```
function revealRow(rowIndex, evaluation) {
  for (let c = 0; c < WORD_LENGTH; c++) {
    const tile = document.getElementById(`tile-${rowIndex}-${c}`);
    tile.dataset.state = evaluation[c];
  }
}
```

No CSS, use seletores por data-state.[code_file:2]

```
.tile[data-state="correct"] { background: #538d4e; border-color: #538d4e; }
.tile[data-state="present"] { background: #b59f3b; border-color: #b59f3b; }
.tile[data-state="absent"] { background: #3a3a3c; border-color: #3a3a3c; }
```

13. código

Criar o teclado virtual

Além do teclado físico, o jogo pode ter um teclado na tela. Isso melhora o uso em celular e deixa o projeto mais parecido com o Termo original.[code_file:2]

Monte o teclado com arrays de letras e gere botões dinamicamente.[code_file:2]

```
const KB_ROWS = [
  ["Q", "W", "E", "R", "T", "Y", "U", "I", "O", "P"],
  ["A", "S", "D", "F", "G", "H", "J", "K", "L"],
  ["ENTER", "Z", "X", "C", "V", "B", "N", "M", "<"]
];
```

Cada botão deve chamar a mesma função de entrada usada pelo teclado físico. Assim você reutiliza a lógica e evita código duplicado.[code_file:2]

14. código

Atualizar as cores do teclado

Depois de cada tentativa, também atualize a cor das teclas já usadas. Assim o jogador consegue lembrar quais letras já testou e o status de cada uma.[code_file:2]

```
function updateKeyboard(evaluation, guess) {
  const priority = { correct: 3, present: 2, absent: 1 };

  guess.split('').forEach((letter, i) => {
    const key = document.querySelector(`[data-key="${letter}"]`);
    const state = evaluation[i];
    const current = key?.dataset.state;

    if (!current || priority[state] > priority[current]) {
      key.dataset.state = state;
    }
  });
}
```

15. código

Criar feedback de vitória e derrota

Depois que a tentativa é enviada, compare o chute com a palavra secreta. Se forem iguais, encerre o jogo com mensagem de vitória. Se o jogador atingir 6 tentativas sem acertar, mostre derrota e revele a resposta.[code_file:2]

```
if (guess === secretWord) {
  gameOver = true;
  showModal(true);
} else if (guesses.length >= MAX_ATTEMPTS) {
  gameOver = true;
  showModal(false);
}
```

16. código

Criar botão de reinício

O botão “Jogar Novamente” deve resetar todas as variáveis e reconstruir o tabuleiro e o teclado.
[code_file:2]

```
function startGame() {  
  secretWord = UNIQUE_WORDS[Math.floor(Math.random() * UNIQUE_WORDS.length)];  
  guesses = [];  
  evaluations = [];  
  currentInput = "";  
  gameOver = false;  
  
  buildBoard();  
  buildKeyboard();  
}
```

Esse reinício evita precisar dar refresh na página.[code_file:2]

17. acabamento

Adicionar animações

Para o jogo ficar mais profissional, adicione pequenas animações no CSS: uma animação ao digitar, outra ao revelar cores e outra para erro de palavra curta.[code_file:2]

As animações usadas no exemplo incluem pop, flipIn, shake e bounce, que deixam a experiência mais parecida com um jogo real.[code_file:2]

18. resultado

Testar o projeto completo

Abra o arquivo index.html no navegador e teste todo o fluxo: digitar letras, apagar, enviar palavras, ganhar, perder e reiniciar a partida.[code_file:2]

Confira também se o jogo funciona tanto com teclado físico quanto com teclado virtual, e se as cores das letras e teclas estão sendo atualizadas corretamente.[code_file:2]

19. organização

Estrutura final esperada

Ao terminar, seu projeto pode ficar organizado assim:[code_file:2]

```
termo/  
├─ index.html
```

```
|— style.css  
|— script.js
```

20. melhoria

O que você pode adicionar depois

Depois que a versão base estiver funcionando, ainda dá para evoluir bastante o projeto.

[code_file:2]

- Lista maior de palavras.
- Verificação de palavras válidas.
- Contador de vitórias.
- Modo claro e escuro.
- Animações melhores.
- Suporte a palavras com acento tratado internamente.
- Layout mais responsivo para celular.